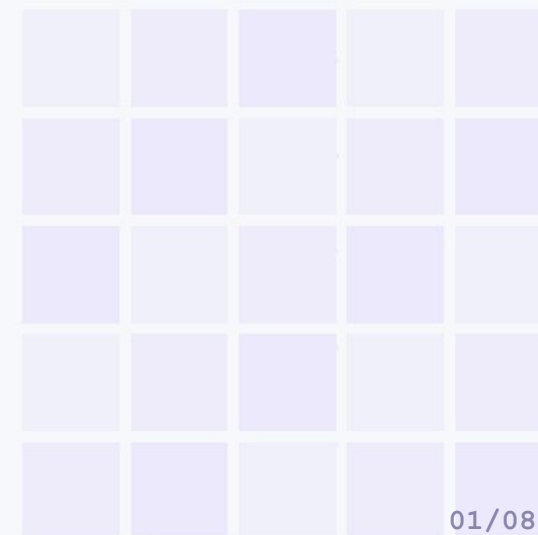


TESSERA

DATA PLATFORM

Descoberta, mascaramento, subsetting e CDC para ambientes não-produtivos rápidos e seguros.

■ VISÃO TÉCNICA DOS MÓDULOS



■ COMO OS MÓDULOS SE ENCAIXAM

VISÃO GERAL DA PLATAFORMA

Dois frameworks de conformidade alimentam dois produtores de dados; um operador orquestra as réplicas.



ORQUESTRAÇÃO Operador de BD (Kubernetes) · roadmap

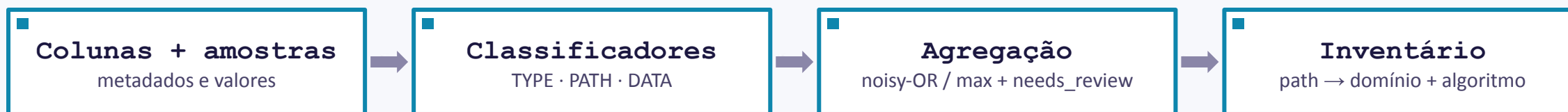
Gerencia réplicas não-produtivas em containers — provision, refresh, snapshot e rollback — sobre Subsetter e CDC.

- Pronto: Discovery · Masking · Subsetter · CDC ● Roadmap: Operador de BD (K8s)

■ FRAMEWORK · BIBLIOTECA GO

DISCOVERY

Classifica colunas em domínios de dados sensíveis e produz um inventário — função pura, sem I/O.



■ **PATH e DATA independentes** cada um classifica sozinho — funciona em esquemas ofuscados (SAP, legado, nomes localizados).

■ **Recall-first** prioriza não deixar coluna sensível passar; match de fonte única vira needs_review automático.

■ **Determinístico** mesmos inputs → mesma classificação, para diffs de inventário estáveis.

■ **Classificadores** type, path (regex), data (regex), luhn, dictionary, entropy — extensível via RegisterClassifierType.

■ **Modelo unificado** relacional e JSON tratados por path[]; sem código separado por formato.

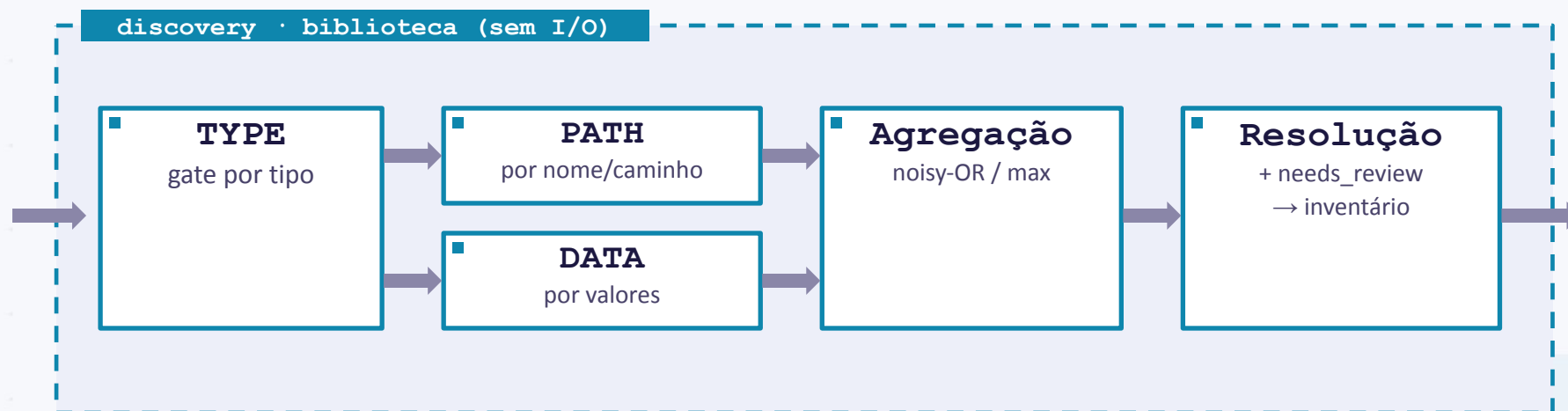
■ **Acurácia medida** harness de corpus rotulado calcula precision/recall/F1 por domínio (regressão empírica).

Contrato de saída: o inventário (path · domínio · algoritmo) é a interface consumida pelo Masking.

■ ARQUITETURA · ALTO NÍVEL

DISCOVERY · ARQUITETURA

PATH e DATA são evidências independentes que convergem para uma classificação.



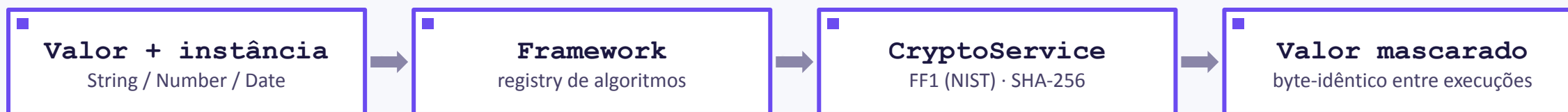
Função pura: o host faz todo o I/O; a integração com o Masking é apenas o arquivo de inventário.

Classificadores plugáveis: type, path, data, luhn, dictionary, entropy...

■ FRAMEWORK · BIBLIOTECA GO

MASKING

Mascaramento determinístico e key-driven — o determinismo é o que preserva a integridade referencial.



■ **Determinismo garantido** mesma chave + config + input → saída byte-idêntica entre processos, máquinas e versões.

■ **FF1 NIST SP 800-38G** implementação validada contra vetores publicados; vetores congelados por algoritmo no CI.

■ **Chaves protegidas** providers de arquivo (dev/K8s Secret) e AWS KMS; a chave nunca sai do engine.

■ **13 algoritmos** Secure Lookup, Character/Numeric/Segment Mapping, Date Shift/Replacement, Full Name, Email...

■ **Formato preservado** Payment Card (Luhn), IBAN (dígito), Mod Checksum (CPF/CNPJ) — bijetivos, formato válido.

■ **Chaining + hot path** algoritmos encadeáveis com detecção de ciclo no load; thread-safe e sem I/O na execução.

Independência: Masking e Discovery não se importam entre si — comunicam-se só pelo formato de inventário.

■ ARQUITETURA · ALTO NÍVEL

MASKING · ARQUITETURA

Framework despacha para o algoritmo; o CryptoService faz a matemática determinística.



Determinismo: mesma chave + config + input → saída byte-idêntica → integridade referencial.

Chave: via arquivo ou AWS KMS, carregada no Setup, nunca sai do engine; hot path sem I/O.

■ APLICAÇÃO · ENTREGA PRINCIPAL

SUBSETTER · ARQUITETURA

Subsets referencialmente íntegros de bancos de produção para dev, teste e staging.



■ **Integridade referencial** planner monta o grafo de dependências e resolve ciclos (Tarjan SCC) em waves; seeds + WHERE.

■ **Masking + Discovery ao vivo** frameworks embarcados aplicam classificação e mascaramento durante a extração.

■ **Grafo de esquema** tabelas, colunas e FKs em Neo4j, com relações lógicas cross-DB ou inferidas.

■ **Determinístico e observável** vetores de determinismo no CI; métricas Prometheus de chamadas de masking.

Um único caminho de execução (engine.RunWithConfig) serve web, CLI e operador — detalhado no próximo slide.

■ APLICAÇÃO · ENTREGA PRINCIPAL

SUBSETTER · SUPERFÍCIES

Três formas de rodar o mesmo motor, plugáveis em qualquer pipeline.

- **WEB**
App React + REST API
 - UI de conectores e grafo
 - Jobs assíncronos + progresso SSE
 - Metadados em PostgreSQL

- **CLI HEADLESS**
subsetter-job / -discover
 - Roda a partir de YAML
 - Sem PostgreSQL/Neo4j
 - Ideal para CI/CD

- **OPERADOR K8s**
CRDs SubsetJob / DiscoveryJob
 - Reconciliação controller-runtime
 - Secrets → credenciais
 - ConfigMaps de masking/inventário

PLATAFORMAS SUPORTADAS

PostgreSQL MySQL SQL Server Oracle MongoDB Object Storage (S3, Azure)

Particionadas: tabelas declarativas do PostgreSQL fluem como unidade lógica com leitura paralela por partição.

DynamoDB está especificado (enum de plataforma), ainda não implementado.

■ ARQUITETURA · ALTO NÍVEL

SUBSETTER · ARQUITETURA

Três superfícies, um único motor; conectores isolam as fontes e o alvo.



Mesmo motor para as três superfícies; os frameworks Masking + Discovery rodam embarcados durante a extração.

Estado: grafo de esquema em Neo4j · metadados em PostgreSQL.

■ APLICAÇÃO · CAPTURA CONTÍNUA

CDC & SNAPSHOT

Snapshot MVCC-consistente + stream incremental ordenado por commit + DDL — para JSONL em S3/FS. Sem Debezium/Kafka.



■ **Sem gap, sem overlap** tudo < LSN0 é do snapshot, ≥ LSN0 é do stream — zero transações perdidas (gap test).

■ **Effectively-once** at-least-once + escritas idempotentes determinísticas; markers no S3 são a fonte de verdade.

■ **Serialização canônica** ordem de campos estável, numéricos como string, ISO-8601, base64 — recuperação byte-idêntica.

■ **Dois motores, paridade** abstração ChangeSource; a suíte de testes do PostgreSQL roda igual no MySQL.

■ **DDL + re-snapshot** ALTER/CREATE/DROP viram registros op=DDL; backpressure com buffer no próprio log.

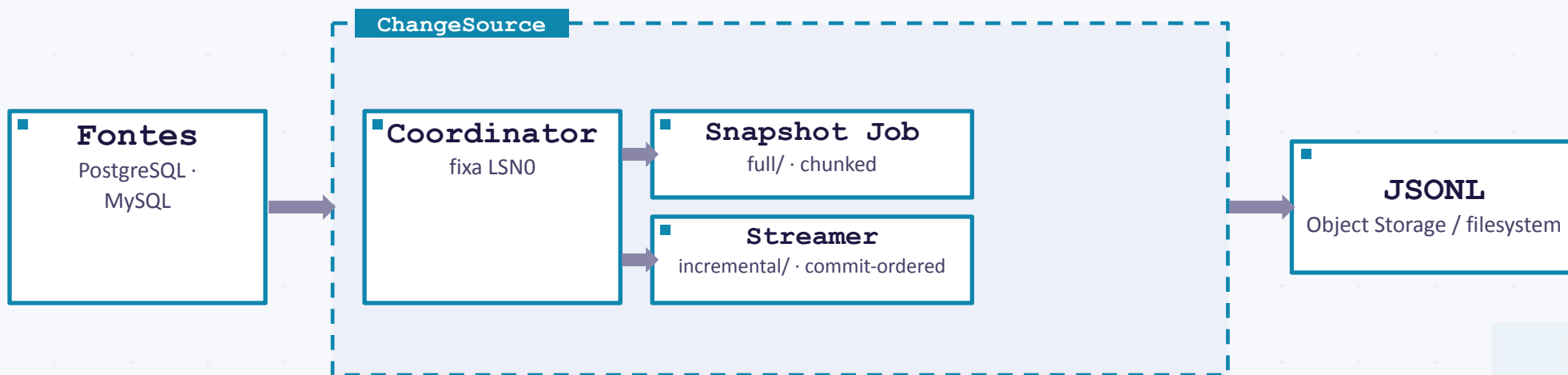
■ **Operador K8s** controller-runtime, 4 CRDs (CaptureSource, ReSnapshot, CaptureDestination, CaptureClass).

RDS/Aurora e self-managed · pglogrepl/pgoutput (sem extensão) · go-mysql (binlog ROW/GTID) · aws-sdk-go-v2

ARQUITETURA · ALTO NÍVEL

CDC & SNAPSHOT · ARQUITETURA

Coordinator fixa o ponto LSN0; Snapshot e Streamer produzem sem gap nem overlap.



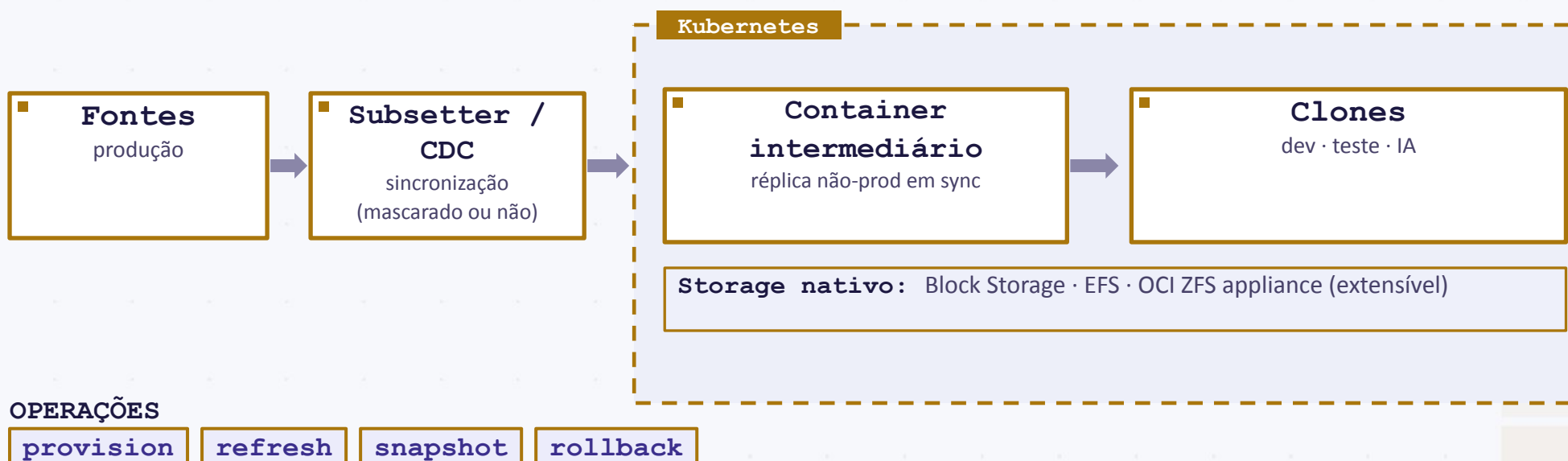
Só compartilham LSN0: tudo < LSN0 é do snapshot, $\geq$ LSN0 é do stream; markers no S3 = fonte de verdade; effectively-once.

Control plane: operador K8s (controller-runtime) — 4 CRDs.

ROADMAP · ARQUITETURA CONCEITUAL

OPERADOR DE BD · KUBERNETES

Lifecycle de réplicas não-produtivas em containers Kubernetes.



⚠ PLACEHOLDER Diagrama conceitual — arquitetura e escopo ainda em definição.